Laboratorio 2

1. Introducción

El parser es el componente que, a partir de una secuencia de *tokens* generados por el analizador léxico, verifica que la expresión cumpla con la gramática definida y construye una representación estructurada, como un árbol sintáctico abstracto (AST). En este laboratorio, implementaremos un parser recursivo descendente, donde cada regla de la gramática se corresponde directamente con una función del programa. La gramática de expresiones aritméticas utilizada se expresa en formato EBNF de la siguiente manera:

```
<program>     ::= <expr> { "+" <expr> }*;

<expr>        ::= <factor> { "*" <factor> }* ;

<factor>      ::= <number>  ;

<number>      ::= <digit> { <digit> }* ;

<digit>       ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9" ;
```

2. Objetivo del laboratorio

- Comprender los conceptos de gramática formal y parser en el contexto del análisis sintáctico de expresiones aritméticas.
- Implementar un parser recursivo descendente capaz de construir un árbol sintáctico abstracto (AST) a partir de una secuencia de tokens.
- Analizar expresiones aritméticas respetando la precedencia de operadores, verificando el correcto funcionamiento del parser.

3. Archivos principales

3.1. Archivo `parser.h`

La clase `Parser` se encarga de realizar el análisis sintáctico de una secuencia de tokens generada por el `Scanner`.

3.1.1. Atributos

- `Scanner* scanner`: Puntero al objeto `Scanner` que proporciona los tokens.
- `Token* current`: Almacena el token actual que se está procesando.
- `Token* previous`: Almacena el token procesado inmediatamente antes del actual.

3.1.2. Métodos principales

- `Parser(Scanner* sc)`: Constructor que inicializa el parser con un scanner, lee el primer token y verifica errores léxicos.
- `bool match(Token::Type ttype)`: Comprueba si el token actual coincide con `ttype`. Si coincide, avanza al siguiente token y devuelve `true`; en caso contrario, devuelve `false`.
- `bool check(Token::Type ttype)`: Verifica si el token actual coincide con `ttype`, sin avanzar.
- `bool advance()`: Avanza al siguiente token, actualiza `previous` y verifica errores léxicos. Devuelve `true` si se avanzó, `false` si se llegó al final.
- `bool isAtEnd()`: Retorna `true` si el token actual indica el final de la entrada (`END`), `false` en caso contrario.
- `Exp* parseProgram()`: Método principal de parseo que inicia el análisis sintáctico y devuelve el AST completo. Lanza un error si no se consume toda la entrada.
- `Exp* parseP()`: Implementa la regla de suma (`P → E ('+' E)*`) y construye nodos `BinaryExp` de suma.
- `Exp* parseE()`: Implementa la regla de multiplicación (`E → F ('*' F)*`) y construye nodos `BinaryExp` de multiplicación.
- `Exp* parseF()`: Implementa la regla de factor (`F → NUM`), creando nodos `NumberExp` con el valor numérico del token.

3.2. Archivo `ast.h`

El archivo `ast.h` define la estructura de datos para el árbol sintáctico abstracto (AST) que será generado por el parser. Incluye clases para representar expresiones numéricas y binarias.

3.2.1. Enumeración de operadores

- `enum BinaryOp`: Define los operadores binarios soportados: `PLUS_OP`, `MINUS_OP`, `MUL_OP` y `DIV_OP`.

3.2.2. Clase abstracta `Exp`

- Clase base para todas las expresiones del AST.
- Métodos:
  - `virtual ~Exp()` : Destructor.
  - `virtual void toDot()` : Método para generar una representación del AST en formato DOT.
  - `static string binopToChar()`: Convierte un operador binario a su representación en string.

3.2.3. Clase `BinaryExp`

- Hereda de `Exp`.
- Atributos:
  - `Exp* left`: puntero a la subexpresión izquierda.
  - `Exp* right`: puntero a la subexpresión derecha.
  - `BinaryOp op`: operador binario aplicado a las subexpresiones..
- Métodos:
  - `BinaryExp(Exp* l, Exp* r, BinaryOp op)`: constructor.
  - `~BinaryExp()`: destructor.
  - `void toDot()`: implementación de la visualización en DOT para la expresión binaria.

3.2.4. Clase `NumberExp`

- Hereda de `Exp`.
- Atributos:
  - `int value`: almacena el valor numérico de la expresión.

- Métodos:
  - `NumberExp(int v)`: constructor.
  - `~NumberExp()`: destructor.
  - `void toDot(std::ostream& out, int& id) const override`: implementación de la visualización en DOT para la expresión numérica.

4. Ejecución del parser

El archivo `main.cpp` implementa el programa principal que utiliza el `Scanner`, el `Parser` y las clases del `AST` para procesar un archivo de entrada con expresiones aritméticas.

4.1. Flujo del programa

1. Verificación de argumentos: El programa espera recibir exactamente un argumento, que corresponde al nombre del archivo de entrada. Si no se proporciona, muestra un mensaje de uso y termina.
2. Lectura del archivo de entrada: Se abre el archivo especificado y se almacena su contenido completo en una variable `string input`.
3. Instanciación del Scanner: Se crean dos objetos `Scanner` a partir del contenido del archivo. Uno se utiliza para imprimir los tokens y otro para el parser.
4. Ejecución del Scanner: Se llama a la función `ejecutar_scanner` para mostrar en pantalla los tokens generados por el primer scanner.
5. Instanciación del Parser: Se crea un objeto `Parser` usando el segundo scanner.
6. Parseo y generación del AST:
   - Se llama a `parser.parseProgram()` para construir el árbol sintáctico abstracto (AST).
   - Se genera un archivo `ast.dot` que contiene la representación del AST en formato DOT.
   - Si ocurre un error léxico o sintáctico, se captura mediante excepciones y se marca el AST como vacío.

4.2. Compilación y ejecución

1. Compilar el programa usando g++:

```
g++  main.cpp scanner.cpp parser.cpp ast.cpp
```

2. Ejecutar el programa proporcionando un archivo de entrada:

```
./a.exe entrada.txt
```

3. Instalar Graphviz (si no está instalado) para visualizar el AST:
   - En Linux: `sudo apt-get install graphviz`
   - En Windows: descargar e instalar desde https://graphviz.org/download/
   - En macOS: `brew install graphviz`
4. Visualizar el AST generado a partir del archivo `ast.dot`:

```
dot -Tpng ast.dot -o ast.png
```

5. Ejercicios

1. Agregar operadores aritméticos: Modificar la gramática y el parser para incluir la resta (`-`) y la división (`/`) en las expresiones.
2. Agregar operador de potencia: Extender el parser para reconocer el operador de potencia (`**`).
3. Soporte para paréntesis: Modificar la función `parseF()` para reconocer expresiones entre paréntesis.
4. Soporte para variables: Extender el parser para reconocer identificadores (variables).
5. Soporte para funciones matemáticas: Implementar el reconocimiento de funciones matemáticas como `sqrt()`, `sin()`, `cos()`, etc.

6. Conclusiones

1. Se comprendió el funcionamiento de un parser recursivo descendente y su papel en la construcción del árbol sintáctico abstracto (AST) a partir de una secuencia de tokens generada por el `Scanner`.

2. Se reforzó la comprensión de conceptos de gramática formal, precedencia de operadores y manejo de errores léxicos y sintácticos mediante excepciones en C++.

3. La implementación del AST permitió visualizar la estructura de las expresiones aritméticas mediante Graphviz, facilitando la comprensión de la jerarquía de operaciones.

4. La práctica de agregar nuevos operadores, paréntesis, variables y funciones matemáticas mostró cómo extender un parser básico para manejar un lenguaje más completo y versátil.

5. Se adquirieron habilidades en la integración de múltiples módulos (scanner, parser, AST y visualización) y en la organización del código para proyectos de análisis de lenguaje.